

Android Process Scheduling — Exercise Set – part 1

1) Consider the processes in the table below:

	PID	nice	Total time (ms)	Weight	DF
P1	1	0	100	1024	1
P2	2	n1 (positive)	80	328	We consider 3.27 but it should be 3.12
P3	3	n2	60	336	3.05

We want to simulate a basic version of the Completely Fair Scheduler where each process gets CPU time according to its "virtual runtime" (vruntime – initially 0), and the process with the smallest vruntime runs next. For each scenario, give the time diagram to show the scheduling of the above processes. **Suppose P3 arrives first to the ready queue followed by P1 and P2.**

- Simulation 1: Fixed time slice. We will use a **fixed actual time slice = 20 ms** for each process.
- Simulation 2: Dynamic time slice. The targeted latency (TL) is 100 ms.
- Simulation 3: Dynamic arrival time with dynamic time slice (TL = 100 ms). At time 90 ms, A new process arrives (PID = 4, nice = 0, tot-time = 40).
- Simulation 4: I/O blocking with dynamic time slice (TL = 100 ms). At time 90 ms, an I/O wait will occur on process with PID = 2. The duration of the wait is 50 ms.

2) Consider the processes in the table below:

	nice	Total time (ms)
P1	0	100
P2	5	80
P3	-5	60

Give the time diagram to show the scheduling of the above processes using CFS with vruntime initially 0.

- Simulation 1: Fixed time slice. We will use a **fixed actual time slice = 20 ms** for each process.
 - Simulation 2: Dynamic time slice. The targeted latency (TL) is 100 ms.
- 3) (M-15/16) Consider the three processes P1, P2, and P3. The process management technique uses CFS with 100 ms of targeted latency. Give the execution order till the end of activities. Justify.

Formulas/Equations: $Vruntime += runtime * DF$ and $Next\ runtime = TS$

Process	Weight	Remaining CPU burst (ms)	Last runtime (ms)
P1	335	60	100
P2	1024	90	200
P3	820	70	200

Important Formulas and Concepts

1. Completely Fair Scheduler (CFS)

The Linux/Android Completely Fair Scheduler (CFS) tries to divide CPU time proportionally according to process weights.

Weight and Nice

- A smaller nice value $\Rightarrow$ larger weight $\Rightarrow$ more CPU share.
- A larger nice value $\Rightarrow$ smaller weight $\Rightarrow$ less CPU share.

Typical weights:

Nice	Weight
-5	3121
0	1024
1	820
5	335

Virtual Runtime

The scheduler chooses the process having the **smallest vruntime**.

Formula:

$$\text{vruntime} += \text{vruntime} \times DF$$

where:

$$DF = \frac{1024}{\text{weight}}$$

Interpretation:

- A high-priority process (large weight) increases vruntime slowly.
 - A low-priority process (small weight) increases vruntime quickly.
-

Dynamic Time Slice in CFS

When target latency is TL:

$$TS_i = TL \times \frac{\text{weight}_i}{\sum \text{weights}}$$

Exercise 1

Processes:

PID	nice	Total CPU	Weight	DF
P1	0	100 ms	1024	1
P2	1	80 ms	328	3.27 (approximated)
P3	2	60 ms	336	3.05

Arrival order:

1. P3 first

2. then P1

3. then P2

Initially:

$$vruntime(P1)=vruntime(P2)=vruntime(P3)=0$$

1.1 Simulation 1 — Fixed Time Slice = 20 ms

The scheduler always executes the process with smallest vruntime.

Step 1

All vruntimes are 0.

Arrival order favors P3 first.

Run P3 for 20 ms.

Update:

$$vruntime(P3)=20 \times 3.05=61$$

Remaining:

- P1 = 100
- P2 = 80
- P3 = 40

Ready queue:

Process	vruntime
P1	0
P2	0
P3	61

Smallest vruntime = P1.

Step 2

Run P1 for 20 ms.

$$vruntime(P1)=20 \times 1=20$$

Remaining:

- P1 = 80
- P2 = 80
- P3 = 40

Queue:

Process	vruntime
P1	20
P2	0
P3	61

Next = P2.

Step 3

Run P2 for 20 ms.

$$vruntime(P2) = 20 \times 3.27 = 65.4$$

Remaining:

- P1 = 80
- P2 = 60
- P3 = 40

Queue:

Process	vruntime
P1	20
P2	65.4
P3	61

P1 has the smallest vruntime.

Step 4

Run P1 for 20 ms.

$$vruntime(P1) = 40$$

Remaining:

- P1 = 60
- P2 = 60
- P3 = 40

Queue:

Process	vruntime
P1	40
P2	65.4
P3	61

Next = P1 again.

Step 5

Run P1 for 20 ms.

$$vruntime(P1)=60$$

Remaining:

- P1 = 40
- P2 = 60
- P3 = 40

Queue:

Process	vruntime
P1	60
P2	65.4
P3	61

Next = P1 again because 60 is smallest.

Step 6

Run P1 for 20 ms.

$$vruntime(P1)=80$$

Remaining:

- P1 = 20
- P2 = 60
- P3 = 40

Queue:

Process	vruntime
P1	80
P2	65.4
P3	61

Next = P3.

Step 7

Run P3 for 20 ms.

$$vruntime(P3)=61+61=122$$

Remaining:

- P1 = 20
- P2 = 60
- P3 = 20

Queue:

Process	vruntime
P1	80
P2	65.4
P3	122

Next = P2.

Step 8

Run P2 for 20 ms.

$$vruntime(P2) = 65.4 + 20 = 85.4$$

Remaining:

- P1 = 20
- P2 = 40
- P3 = 20

Next = P1.

Step 9

Run P1 for last 20 ms.

$$vruntime(P1) = 100$$

P1 finishes.

Remaining:

- P2 = 40
- P3 = 20

Next = P3 because $122 < 130.8$.

Step 10

Run P3 for final 20 ms.

$$vruntime(P3) = 142$$

P3 finishes.

Step 11

Run P2 for 20 ms.

$$vruntime(P2) = 130.8 + 65.4 = 196.2$$

Remaining:

- P2 = 20
-

Step 12

Run P2 final 20 ms.

P2 finishes.

Final Timeline

0	20	40	60	80	100	120	140	160	180	200	220	240
P3	P1	P2	P1	P1	P1	P3	P2	P1	P3	P2	P2	

1.2 Simulation 2 — Dynamic Time Slice (TL = 100 ms)

Total weight:

$$1024 + 328 + 336 = 1688$$

Time slices:

For P1:

$$T S_1 = 100 \times \frac{1024}{1688} \approx 60.8 \text{ ms}$$

For P2:

$$T S_2 = 100 \times \frac{328}{1688} \approx 19.3 \text{ ms}$$

For P3:

$$T S_3 = 100 \times \frac{336}{1688} \approx 19.9 \text{ ms}$$

Interpretation:

- P1 gets most CPU time.
 - P2 and P3 receive smaller slices.
-

First Round

P3 executes first

Runs 19.9 ms.

$$vruntime(P3) = 19.9 \times 3.05 = 60.7$$

Remaining: $60 - 19.9 = 40.1\text{ ms}$

P1 executes

Runs 60.8 ms.

$$vruntime(P1) = 60.8$$

Remaining: $100 - 60.8 = 39.2\text{ ms}$

P2 executes

Runs 19.3 ms.

$$vruntime(P2) = 19.3 \times 3.27 = 63.11$$

Remaining: $80 - 19.11 = 60.89\text{ ms}$

Now:

Process	vruntime
P1	60.8
P2	63.11
P3	60.7

Smallest vruntime is actually P3. Therefore P3 must execute next, followed by P1, then P2.

Second Round

P3 runs another 19.9 ms.

$$vruntime(P3) = 121.4$$

Remaining: 20.2 ms

P1 executes next and runs its remaining 39.2 ms.

$$vruntime(P1) = 100$$

Remaining: *P1 finishes execution*

Next = P2.

P2 runs another 19.3 ms.

$$vruntime(P2) = 63.11 + 19.3 \times 3.27 = 126.22$$

Remaining: 41.4 ms

Process	vruntime
P2	126.22
P3	121.4

P3 is selected then P2.

Remaining Round

P3 runs remaining 20.2 ms and finishes.

P2 runs remaining 41.4 ms and finishes.

Timeline

0	-	19.9	-	80.7	-	100	-	119.9	-	159.1	-	178.4	-	198.6	-	240
	P3		P1		P2		P3		P1		P2		P3		P2	

Exercise 2

Processes:

Process	nice	Total CPU
P1	0	100
P2	5	80
P3	-5	60

Corresponding weights:

Process	Weight	DF
P1	1024	1
P2	336	3.05
P3	3125	0.33

Total weight:

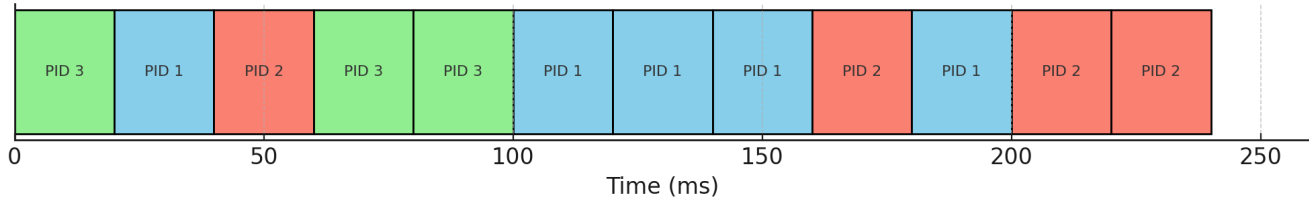
$$1024 + 336 + 3125 = 4485$$

Simulation 1: Static Slices

P3 has the highest priority (nice is -5).

Time	Action	PID	Vruntime updated to
0	Run PID 3 (first time)	3	6.6
20	Run PID 1 (first time)	1	20
40	Run PID 2 (first time)	2	61
60	Run PID 3 (second time)	3	13.2
80	Run PID 3 (third time, finishes)	3	19.8 (finished)
100	Run PID 1	1	40
120	Run PID 1	1	60
140	Run PID 1	1	80
160	Run PID 2	2	122
180	Run PID 1	1	100 (finished)
200	Run PID 2	2	183
220	Run PID 2	2	244(finished)

CFS Scheduling Timeline with vruntime-based Decisions



Simulation 2: Dynamic Slices

For P1:

$$T S_1 = 100 \times \frac{1024}{4485} = 22.83 \text{ ms}$$

For P2:

$$T S_2 = 100 \times \frac{336}{4485} = 7.49 \text{ ms}$$

For P3:

$$T S_3 = 100 \times \frac{3125}{4485} = 69.68 \text{ ms}$$

Observation:

- P3 has extremely high priority.
- P3 dominates the CPU.
- P2 receives very little CPU time.

Execution Analysis

Initially all vruntime = 0.

Since P3 has huge weight, its vruntime grows very slowly:

$$DF_{P3} = \frac{1024}{3125} = 0.33$$

Thus after 60ms will terminate from the first round:

$$vruntime(P3) = 60 \times 0.33 \approx 19.8$$

For P1:

$$DF_{P1} = \frac{1024}{1024} = 1$$

Thus after 22.83 ms:

$$vruntime(P1) = 22.83$$

For P2:

$$DF_{P2} = \frac{1024}{336} = 3.05$$

Thus after 7.49 ms:

$$vruntime(P2) = 22.84$$

But here in round 2, we have just two processes:

For P1:

$$TS_1 = 100 \times \frac{1024}{1360} = 75.3 \text{ ms}$$

For P2:

$$TS_2 = 100 \times \frac{336}{1360} = 24.7 \text{ ms}$$

...

Step-by-Step Execution

This section details each scheduling decision made based on vruntime values:

Round 1

- Time 0ms: PID 3 runs (60ms), $vruntime_3 += 19.8 \rightarrow 19.8$ (finished)
- Time 60ms: PID 1 runs (22.83ms), $vruntime_1 += 22.83 \rightarrow 22.83$ (smallest)
- Time 82.83ms: PID 2 runs (7.49ms), $vruntime_2 += 22.84 \rightarrow 22.84$

Round 2

- Time 90.32ms: PID 1 runs (75.3ms), $vruntime_1 += 75.3 \rightarrow 98.13$ (smallest)
- Time 165.62ms: PID 2 runs (24.7ms), $vruntime_2 += 75.34 \rightarrow 98.18$

Round 3

- Time 190.32ms: PID 1 runs (1.87ms), $vruntime_1 += 22.86 \rightarrow 45.72$ (finished)
- Time 192.19ms: PID 2 runs (24.7ms), $vruntime_2 += 75.34 \rightarrow 173.52$

Round 4 (P2 alone and can have the complete TL)

- Time 216.89ms: PID 2 runs (23.11ms), $vruntime_2 += 70.49 \rightarrow 244.01$ (finished)
- Time 240ms: All finished

Exercise 3 — CFS Execution Order

Given:

Process	Weight	Remaining Burst	Last Runtime
P1	335	60	100
P2	1024	90	200
P3	820	70	200

Target latency = 100 ms.

Step 1 — Compute $vruntime$

Formula:

$$vruntime = runtime \times DF$$

where:

$$DF = \frac{1024}{weight}$$

P1

$$DF_1 = \frac{1024}{335} = 3.056$$

$$vruntime_1 = 100 \times 3.056 = 305.6$$

P2

$$DF_2 = 1$$

$$vruntime_2 = 200$$

P3

$$DF_3 = \frac{1024}{820} = 1.248$$

$$vruntime_3 = 200 \times 1.248 = 249.6$$

Ordering

Smallest runtime runs first:

Process	runtime
P2	200
P3	249.6
P1	305.6

Therefore:

Execution order within the first round (100 ms):

P2 → P3 → P1

with $TS(i) = TL * W(i) / ToT(Wj)$ where $ToT(Wj) = 335+1024+820 = 2179$

So, $TS1 = 15.37$ $TS2 = 47$ $TS3 = 37.63$

After first round 1, runtime is updated per process and we have:

$runtime1 += 46.97 = 305.6 + 46.97 = 352.57$

$runtime2 += 47 = 200 + 47 = 247$

$runtime3 += 46.96 = 249.6 + 46.96 = 296.56$

Again, Execution order within the first round (100 ms):

P2 → P3 → P1

With

$Remain2 = 90 - 47 = 43$,

$Remain3 = 70 - 37.63 = 32.37$,

$Remain1 = 60 - 15.37 =$

So, $TS1 = 15.37$ $TS2 = 43$ (of 47) $TS3 = 32.37$ (of 37.63)

in Round 2, both P2 and P3 will finish and P1 continue till ending..

$0 - 47 - 84.63 - 100 - 143 - 175.37 - 190.74 - 220$

P2	P3	P1	P2	P3	P1	P1	
----	----	----	----	----	----	----	--